

Berikut adalah penjelasan mendalam mengenai kode C# tersebut. Kode ini berfungsi sebagai **eksportir data gambar**, di mana ia membaca setiap piksel dari sebuah gambar (Bitmap) dan menyimpan informasi koordinat serta warnanya (RGB) ke dalam sebuah file teks.

Berikut adalah rincian per bagian dan per baris kodenya:

1. Bagian Import (Using Directives)

C#

```
using System.Drawing.Drawing2D;  
using System.Text;  
using System.IO;  
using System;
```

- **using System.Drawing.Drawing2D;**: Menyediakan fungsionalitas grafis lanjutan. Namun, pada kode spesifik ini, library utamanya adalah System.Drawing (untuk kelas Bitmap dan Color), yang biasanya sudah terimplikasi atau perlu ditambahkan secara eksplisit.
- **using System.Text;**: Digunakan untuk mengakses kelas StringBuilder. Ini sangat penting untuk efisiensi memori saat menggabungkan string dalam jumlah banyak.
- **using System.IO;**: Digunakan untuk operasi Input/Output file, khususnya untuk menulis file ke disk (File.WriteAllText).
- **using System;**: Library dasar C# untuk tipe data dasar dan *Exception* (seperti ArgumentException).

2. Namespace dan Definisi Kelas

C#

```
namespace MiniPhotoshop.Logic.IO  
{  
    public static class FileExporter  
    {
```

- **namespace MiniPhotoshop.Logic.IO:** Ini adalah "alamat" atau pengelompokan kode.

Kode ini mengindikasikan bahwa kelas ini adalah bagian dari logika Input/Output (IO) untuk aplikasi bernama "MiniPhotoshop".

- **public static class FileExporter:**
 - **public:** Kelas ini bisa diakses dari mana saja dalam proyek.
 - **static:** Anda tidak perlu membuat objek baru (instansiasi) dengan `new FileExporter()` untuk menggunakannya. Anda bisa langsung memanggil fungsinya.
 - **FileExporter:** Nama kelas yang deskriptif, menunjukkan tugasnya untuk mengekspor file.
-

3. Definisi Metode (Fungsi Utama)

C#

```
public static void ExportPixelDataToTxt(Bitmap bmp, string filePath)
{
```

- **public static void:** Fungsi ini dapat diakses publik, bersifat statis (milik kelas, bukan objek), dan tidak mengembalikan nilai apa pun (void).
 - **Parameter:**
 - `Bitmap bmp`: Objek gambar yang akan dibaca. `Bitmap` merepresentasikan gambar yang tersimpan di memori sebagai grid piksel.
 - `string filePath`: Lokasi lengkap (path) di mana file teks hasil ekspor akan disimpan (contoh: "C:\Data\pixeldata.txt").
-

4. Validasi (Guard Clause)

C#

```
if (bmp == null)
{
    throw new ArgumentNullException(nameof(bmp), "Bitmap tidak boleh null.");
}
```

- **Tujuan:** Mencegah program *crash* (berhenti mendadak) jika pengguna lupa memasukkan gambar atau mengirim data kosong.

- **throw new ArgumentNullException(...):** Melemparkan pesan error yang jelas jika bmp kosong, memberi tahu programmer bahwa argumen bmp adalah penyebabnya.
-

5. Inisialisasi StringBuilder

C#

```
var sb = new StringBuilder();
```

- **Mengapa tidak pakai string biasa?**
 - Jika Anda menggunakan string biasa di dalam loop (misal: teks += "data"), komputer akan membuat objek string baru di memori setiap kali loop berjalan. Jika gambar berukuran 1000x1000 piksel (1 juta piksel), komputer akan terbebani dan menjadi sangat lambat.
 - **StringBuilder** dirancang untuk memanipulasi string yang sering berubah dengan sangat cepat dan efisien.
-

6. Loop Iterasi Piksel (Inti Logika)

C#

```
for (int y = 0; y < bmp.Height; y++)  
{  
    for (int x = 0; x < bmp.Width; x++)  
    {
```

- **Nested Loop (Loop Bersarang):** Kita menggunakan dua perulangan for untuk menjelajahi setiap titik pada gambar.
 - **y (Baris/Vertikal):** Loop luar mengontrol koordinat Y (dari atas ke bawah).
 - **x (Kolom/Horizontal):** Loop dalam mengontrol koordinat X (dari kiri ke kanan).
 - **Alur:** Kode akan membaca baris ke-0 dari kiri ke kanan, lalu pindah ke baris ke-1, dan seterusnya (seperti cara kita membaca buku).
-

7. Mengambil dan Memformat Data Piksel

C#

```
        Color pixelColor = bmp.GetPixel(x, y);
        sb.Append($"{x},{y}: R={pixelColor.R}, G={pixelColor.G}, B={pixelColor.B}\n");
    }
}
```

- **bmp.GetPixel(x, y)**: Mengambil warna pada koordinat (x, y). Ini mengembalikan objek Color yang berisi komponen Merah (R), Hijau (G), Biru (B), dan Alpha (A).
- **sb.Append(...)**: Menambahkan string baru ke dalam memori StringBuilder.
- **(\$"{x},{y}: ...")**: Ini adalah *String Interpolation*. Tanda \$ memungkinkan kita memasukkan variabel langsung ke dalam string menggunakan kurung kurawal {}.
- **\n**: Karakter *New Line* (baris baru), agar data setiap piksel ditulis di baris yang berbeda di file teks nanti.

8. Menulis ke File

C#

```
        // Memastikan penggunaan System.IO
        File.WriteAllText(filePath, sb.ToString());
    }
}
```

- **sb.ToString()**: Mengubah semua data yang sudah dikumpulkan oleh StringBuilder menjadi satu string utuh yang sangat panjang.
- **File.WriteAllText(filePath, ...)**: Membuat file baru di lokasi filePath (atau menimpa jika sudah ada) dan menuliskan seluruh isi string ke dalamnya sekaligus. Ini otomatis menangani pembukaan dan penutupan stream file.

Ringkasan Kinerja (Performance Note)

Kode ini **bagus untuk pemula** atau untuk gambar yang sangat kecil (misal ikon 32x32). Namun, ada satu kelemahan teknis:

Metode GetPixel itu lambat.

Jika Anda mencoba mengeksport foto HD (misal 1920x1080 = 2 juta piksel), fungsi GetPixel akan dipanggil 2 juta kali, dan proses ini memakan waktu cukup lama karena GetPixel melakukan pengecekan keamanan memori setiap kali dipanggil.

Untuk aplikasi profesional ("MiniPhotoshop" yang lebih cepat), biasanya programmer menggunakan teknik **LockBits** (mengunci memori dan membaca *byte array* secara langsung) yang jauh lebih cepat daripada GetPixel.

Langkah selanjutnya yang bisa saya lakukan untuk Anda:

Apakah Anda ingin melihat versi kode yang lebih canggih dan cepat menggunakan metode LockBits agar proses ekspor gambar besar tidak membuat aplikasi macet?